Ohm Improvement

Programming for Drift Tolerance in Computational Phase-Change Memory

Peter Coffman | November 4, 2026

AI is expensive

AI is expensive

Power



Photo: Jakec; edit: Crisco 1492 / Wikimedia Commons / CC BY-SA 4.0.

AI is expensive

Power



Photo: Jakec; edit: Crisco 1492 / Wikimedia Commons / CC BY-SA 4.0.

Hardware



Photo: Carlos Jones/ORNL / CC BY 2.0.

AI is expensive

Power



Photo: Jakec; edit: Crisco 1492 / Wikimedia Commons / CC BY-SA 4.0.

Hardware



Photo: Carlos Jones/ORNL / CC BY 2.0.

Time



Screenshot: OpenAI ChatGPT, captured by the author.

Why is AI expensive?

Why is AI expensive?

Mostly one operation: *Matrix-Vector Multiplication (MVM)*

Why is AI expensive?

Mostly one operation: *Matrix-Vector Multiplication (MVM)*

$$(2 \times 3) + (4 \times 1) = 10$$
$$(1 \times 3) + (3 \times 1) = 6$$

Why is AI expensive?

Mostly one operation: *Matrix-Vector Multiplication (MVM)*

$$(2 \times 3) + (4 \times 1) = 10$$
$$(1 \times 3) + (3 \times 1) = 6$$

$$\underbrace{\begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}}_{\text{stored weights}} \quad \underbrace{\begin{bmatrix} 3 \\ 1 \end{bmatrix}}_{\text{inputs}} = \underbrace{\begin{bmatrix} 10 \\ 6 \end{bmatrix}}_{\text{outputs}}$$

4 stored weights

Why is AI expensive?

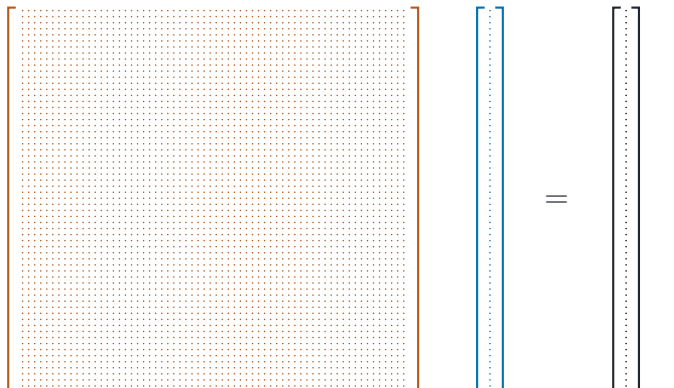
$$(1 \times 3) + (0 \times 5) + (4 \times 5) + (3 \times 2) + (3 \times 2) + (2 \times 3) + (1 \times 2) + (8 \times 5) + (1 \times 5) + (9 \times 1) + (6 \times 5) + (0 \times 3) = 127$$

$$\begin{bmatrix} 1 & 0 & 4 & 3 & 3 & 2 & 1 & 8 & 1 & 9 & 6 & 0 \\ 0 & 1 & 3 & 3 & 8 & 9 & 0 & 8 & 3 & 8 & 6 & 3 \\ 7 & 9 & 4 & 0 & 2 & 6 & 5 & 4 & 2 & 3 & 5 & 1 \\ 1 & 6 & 1 & 5 & 5 & 9 & 4 & 0 & 7 & 8 & 1 & 6 \\ 1 & 8 & 4 & 9 & 5 & 9 & 3 & 1 & 0 & 3 & 4 & 1 \\ 3 & 1 & 6 & 4 & 7 & 5 & 2 & 5 & 5 & 3 & 4 & 1 \\ 9 & 2 & 8 & 3 & 2 & 7 & 6 & 4 & 8 & 3 & 5 & 0 \\ 3 & 0 & 5 & 6 & 4 & 1 & 3 & 9 & 5 & 3 & 7 & 6 \\ 7 & 2 & 4 & 2 & 3 & 8 & 8 & 4 & 9 & 6 & 9 & 6 \\ 5 & 3 & 2 & 8 & 7 & 1 & 0 & 1 & 2 & 2 & 6 & 9 \\ 1 & 6 & 6 & 9 & 7 & 8 & 4 & 8 & 0 & 1 & 8 & 4 \\ 5 & 1 & 4 & 6 & 2 & 7 & 0 & 4 & 8 & 2 & 8 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 5 \\ 5 \\ 2 \\ 2 \\ 3 \\ 2 \\ 5 \\ 5 \\ 1 \\ 5 \\ 3 \end{bmatrix} = \begin{bmatrix} 127 \\ 171 \\ 179 \\ 159 \\ 155 \\ 161 \\ 208 \\ 189 \\ 235 \\ 147 \\ 220 \\ 182 \end{bmatrix}$$

stored weights inputs outputs

144 stored weights

Why is AI expensive?



4,096 stored weights

At this scale, an AI weight matrix could be about
65 metres wide.

$8{,}192 \times 8{,}192$ weights, on a 72-inch display.

At this scale, an AI weight matrix could be about
65 metres wide.

$8{,}192 \times 8{,}192$ weights, on a 72-inch display.

AI models can have *trillions* of weights,
requiring many trillions of operations per response.

At this scale, an AI weight matrix could be about
65 metres wide.

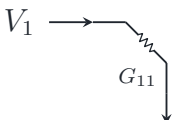
$8{,}192 \times 8{,}192$ weights, on a 72-inch display.

AI models can have *trillions* of weights,
requiring many trillions of operations per response.

How can we make MVM cheaper?

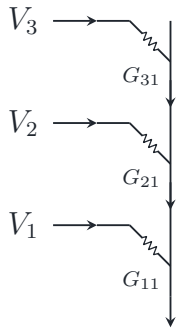
MVM from circuit laws

MVM from circuit laws



$I_1 = V_1 G_{11}$
(Ohm's Law)

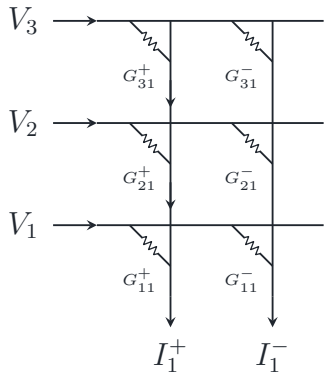
MVM from circuit laws



$$I_1 = V_1 G_{11} + V_2 G_{21} + V_3 G_{31}$$

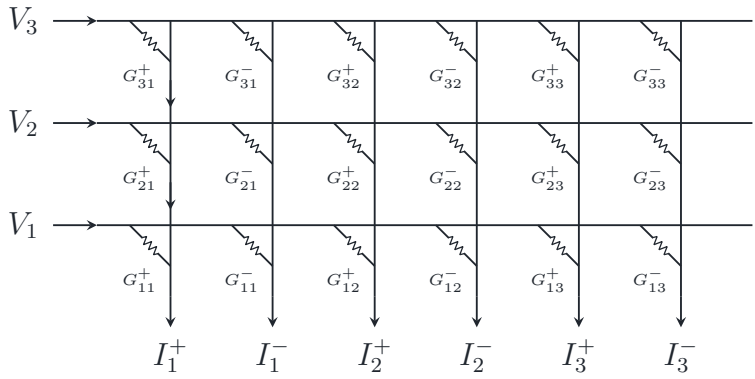
(Kirchhoff's Current Law)

MVM from circuit laws



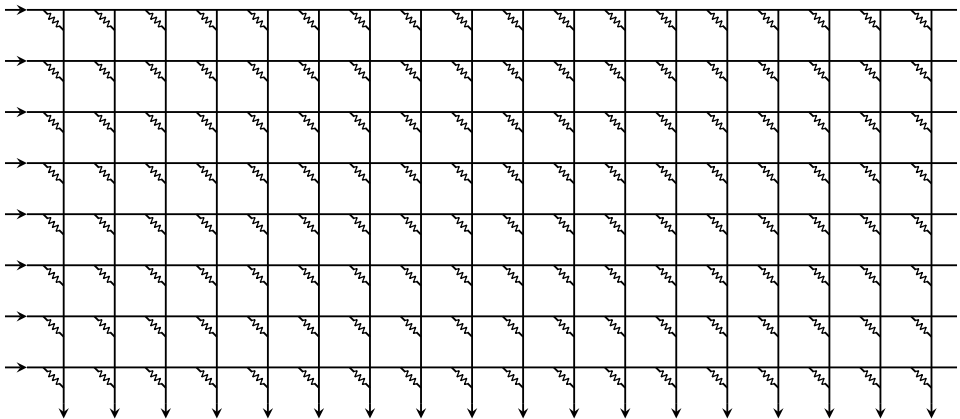
$I_{\text{diff}} = I_1^+ - I_1^-$
(Differential pairs)

MVM from circuit laws

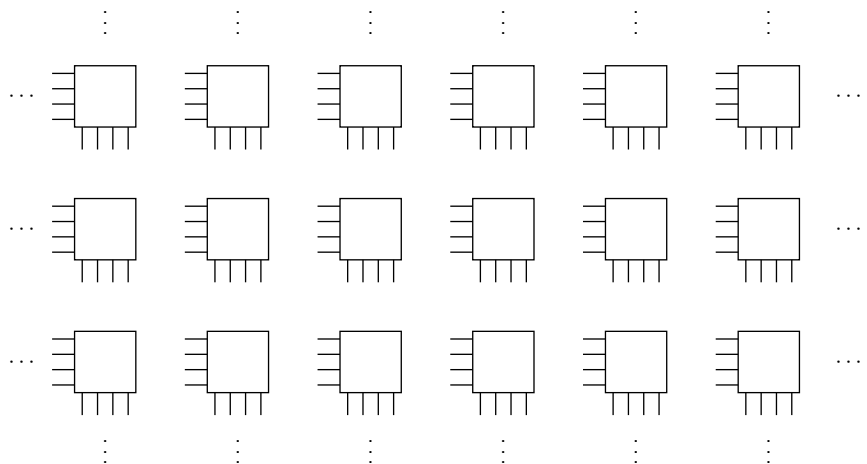


$$I_j = V_1(G^+_{1j} - G^-_{1j}) + V_2(G^+_{2j} - G^-_{2j}) + V_3(G^+_{3j} - G^-_{3j})$$

MVM from circuit laws



MVM from circuit laws



Phase-change memory

- One weight stored as one conductance

Phase-change memory

- One weight stored as one conductance
- Programmed with *current pulses*

Phase-change memory

- One weight stored as one conductance
- Programmed with *current pulses*
- Several *nonidealities*

Phase-change memory

- One weight stored as one conductance
- Programmed with *current pulses*
- Several *nonidealities*
  - Programming noise

Phase-change memory

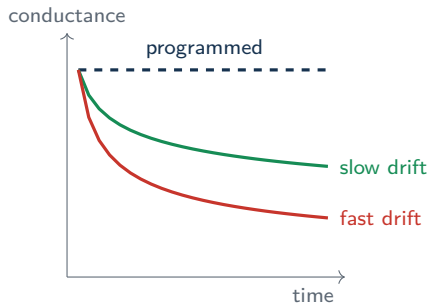
- One weight stored as one conductance
- Programmed with *current pulses*
- Several *nonidealities*
  - Programming noise
  - Read noise

Phase-change memory

- One weight stored as one conductance
- Programmed with *current pulses*
- Several *nonidealities*
  - Programming noise
  - Read noise
  - Device variation

Phase-change memory

- One weight stored as one conductance
- Programmed with *current pulses*
- Several *nonidealities*
  - Programming noise
  - Read noise
  - Device variation
  - Drift— $G(t) = G(t_0)\left(\frac{t}{t_0}\right)^{-\nu}$



Tolerating drift

- Different ways of representing $\vec{y} = A\vec{x} = (A^+ - A^-)\vec{x}$

Tolerating drift

- Different ways of representing $\vec{y} = A\vec{x} = (A^+ - A^-)\vec{x}$

Offset weights on differential pairs	$\vec{y} = [(A^+ + B) - (A^- + B)]\vec{x}$

Tolerating drift

- Different ways of representing $\vec{y} = A\vec{x} = (A^+ - A^-)\vec{x}$

Offset weights on differential pairs	$\vec{y} = [(A^+ + B) - (A^- + B)]\vec{x}$
Permutations of rows and columns	$\vec{y} = P^{-1}AP\vec{x}$

Tolerating drift

- Different ways of representing $\vec{y} = A\vec{x} = (A^+ - A^-)\vec{x}$

Offset weights on differential pairs	$\vec{y} = [(A^+ + B) - (A^- + B)]\vec{x}$
Permutations of rows and columns	$\vec{y} = P^{-1}AP\vec{x}$
Scaled weights with compensation	$\vec{y} = \frac{1}{s}(sA)\vec{x}$

Tolerating drift

- Different ways of representing $\vec{y} = A\vec{x} = (A^+ - A^-)\vec{x}$

Offset weights on differential pairs	$\vec{y} = [(A^+ + B) - (A^- + B)]\vec{x}$
Permutations of rows and columns	$\vec{y} = P^{-1}AP\vec{x}$
Scaled weights with compensation	$\vec{y} = \frac{1}{s}(sA)\vec{x}$

- These diverge under drift.

Tolerating drift

- Different ways of representing $\vec{y} = A\vec{x} = (A^+ - A^-)\vec{x}$

Offset weights on differential pairs	$\vec{y} = [(A^+ + B) - (A^- + B)]\vec{x}$
Permutations of rows and columns	$\vec{y} = P^{-1}AP\vec{x}$
Scaled weights with compensation	$\vec{y} = \frac{1}{s}(sA)\vec{x}$

- These diverge under drift.
- Pick B, P, s to minimize error over time.

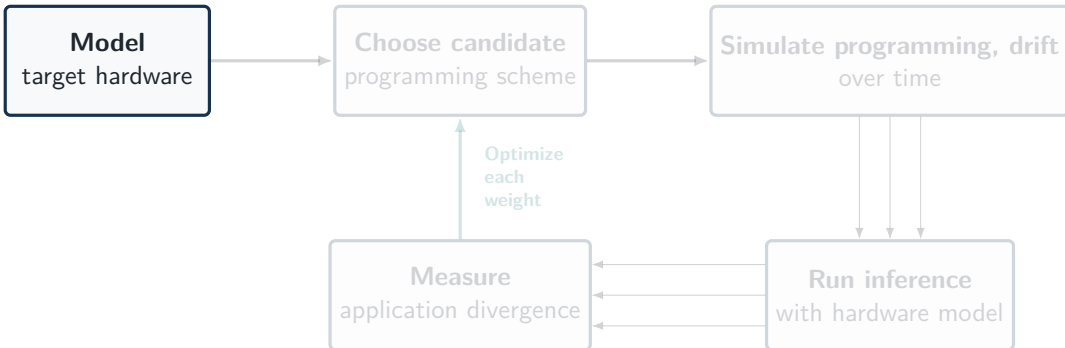
Tolerating drift

- Different ways of representing $\vec{y} = A\vec{x} = (A^+ - A^-)\vec{x}$

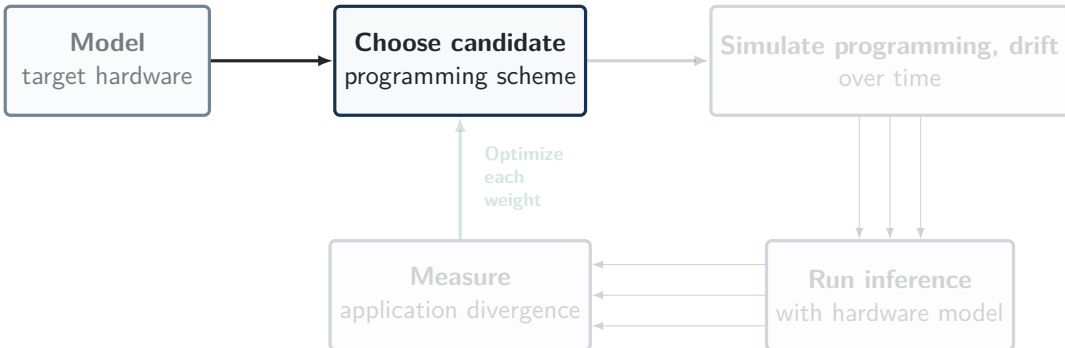
Offset weights on differential pairs	$\vec{y} = [(A^+ + B) - (A^- + B)]\vec{x}$
Permutations of rows and columns	$\vec{y} = P^{-1}AP\vec{x}$
Scaled weights with compensation	$\vec{y} = \frac{1}{s}(sA)\vec{x}$

- These diverge under drift.
- Pick B, P, s to minimize error over time.
- **More general**: optimize each conductance individually

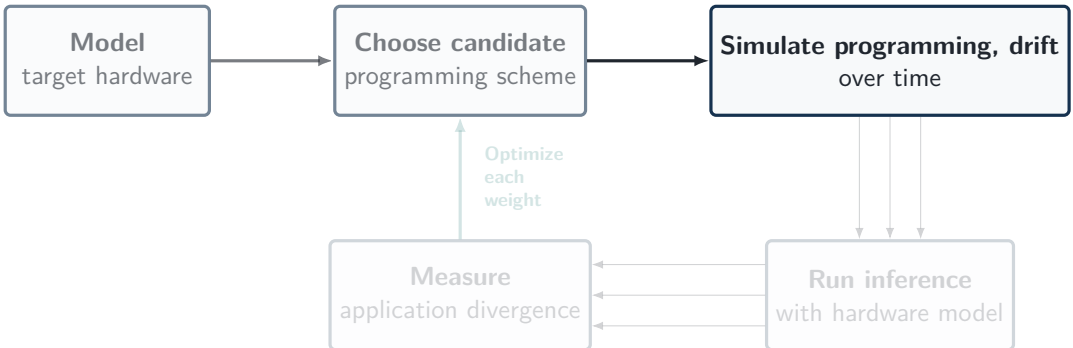
Optimization loop



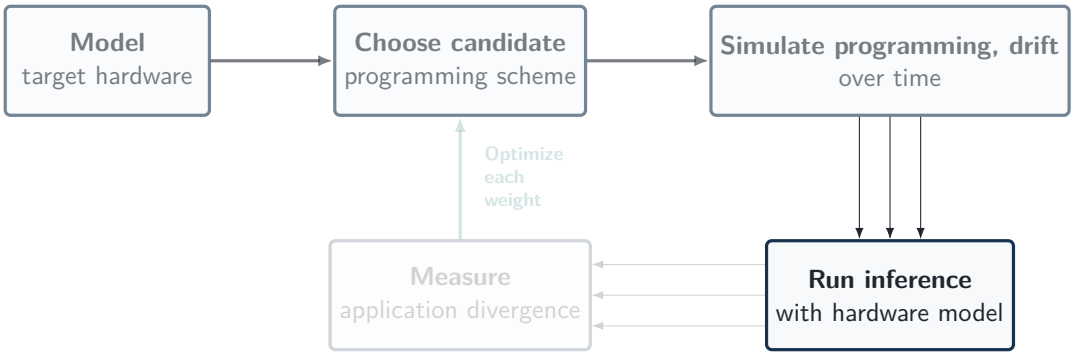
Optimization loop



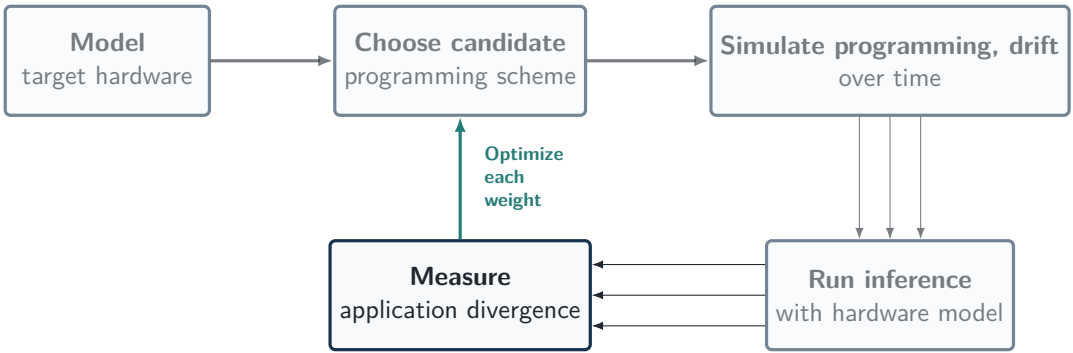
Optimization loop



Optimization loop



Optimization loop



Optimization loop

